

Singleton design pattern in java

Singleton pattern is a design solution where an application wants to have one and only one instance of any class, in all possible scenarios without any exceptional condition. It has been debated long enough in java community regarding possible approaches to make any class singleton. Still, you will find people not satisfied with any solution you give. They can not be overruled either. In this post, we will discuss some good approaches and will work towards our best possible effort.

Sections in this post:

- Eager initialization
- Lazy initialization
- Static block initialization
- Bill pugh solution
- Using Enum
- Adding readResolve()
- Adding serial version id
- Conclusion

Singleton term is derived from its mathematical counterpart. It wants us, as said above, to have only one instance. Lets see the possible solutions:

Eager initialization

This is a design pattern where an instance of a class is created much before it is actually required. Mostly it is done on system start up. In singleton pattern, it refers to create the singleton instance irrespective of whether any other class actually asked for its instance or not.

```

1  public class EagerSingleton {
2      private static volatile EagerSingleton instance = new EagerSingleton();
3
4      // private constructor
5      private EagerSingleton() {
6      }
7
8
9
10     public synchronized static EagerSingleton getInstance() {
11         if (instance == null) {
12             instance = new EagerSingleton();
13         }
14         return instance;
15     }
16 }

```

Above method works fine, but has one performance drawback. The `getInstance()` method is synchronized and each call will require extra locking/unlocking steps which are necessary only for first time, and never there after.

Lets solve above problem in next method.

Lazy initialization

In computer programming, lazy initialization is the tactic of delaying the creation of an object, the calculation of a value, or some other expensive process until the first time it is needed. In singleton pattern, it restricts the creation of instance until requested first time. Lets see in code:

```

1  public final class LazySingleton {
2      private static volatile LazySingleton instance = null;
3
4      // private constructor
5      private LazySingleton() {
6      }
7
8
9
10     public static LazySingleton getInstance() {
11         if (instance == null) {
12             synchronized (LazySingleton.class) {
13                 instance = new LazySingleton();
14             }
15         }
16         return instance;
17     }
18 }

```

On first invocation, above method will check if instance is already created using instance variable. If there is no instance i.e. instance is null, it will create an instance and will return its reference. If instance is already created, it will simply return the reference of instance.

But, this method also has its own drawbacks. Lets see how. Suppose there are two threads T1 and T2. Both comes to create instance and execute "instance==null", now both threads have identified instance variable to null thus assume they must create an instance. They sequentially goes to synchronized block and create the instances. At the end, we have two instances in our application.

This error can be solved using double-checked locking. This principle tells us to recheck the instance variable again in synchronized block in given below way:

```
1  public class EagerSingleton {
2      private static volatile EagerSingleton instance = null;
3
4      // private constructor
5      private EagerSingleton() {
6      }
7
8
9      public static EagerSingleton getInstance() {
10         if (instance == null) {
11             synchronized (EagerSingleton.class) {
12                 // Double check
13                 if (instance == null) {
14                     instance = new EagerSingleton();
15                 }
16             }
17         }
18         return instance;
19     }
20 }
```

Above code is the correct implementation of singleton pattern.

Please ensure to use "volatile" keyword with instance variable otherwise you can run into out of order write error scenario, where reference of instance is returned before actually the object is constructed i.e. JVM has only allocated the memory and constructor code is still not executed. In this case, your other thread, which refer to uninitialized object may throw null pointer exception and can even crash the whole application.

Static block initialization

If you have little idea about class loading sequence, you can connect to the fact that static blocks are executed during the loading of class and even before the constructor is called. We can use this feature in

our singleton pattern also like this:

```
1  public class StaticBlockSingleton {
2      private static final StaticBlockSingleton INSTANCE;
3
4      static {
5          try {
6              INSTANCE = new StaticBlockSingleton();
7          } catch (Exception e) {
8              throw new RuntimeException("Uffff, i was not expecting this!", e);
9          }
10     }
11
12     public static StaticBlockSingleton getInstance() {
13         return INSTANCE;
14     }
15
16     private StaticBlockSingleton() {
17         // ...
18     }
19 }
```

Above code has one drawback. Suppose there are 5 static fields in class and application code needs to access only 2 or 3, for which instance creation is not required at all. So, if we use this static initialization. we will have one instance created though we require it or not.

Next section will overcome this problem.

Bill pugh solution

Bill pugh was main force behind java memory model changes. His principle "Initialization-on-demand holder idiom" also uses static block but in different way. It suggest to use static inner class.

```

1  public class BillPughSingleton {
2      private BillPughSingleton() {
3      }
4
5
6      private static class LazyHolder {
7          private static final BillPughSingleton INSTANCE = new BillPughSingleton()
8      }
9
10
11     public static BillPughSingleton getInstance() {
12         return LazyHolder.INSTANCE;
13     }
14 }

```

As you can see, until we need an instance, the LazyHolder class will not be initialized until required and you can still use other static members of BillPughSingleton class. *This is the solution, i will recommend to use. I also use it in my all projects.*

Using Enum

This type of implementation recommend the use of enum. Enum, as written in java docs, provide implicit support for thread safety and only one instance is guaranteed. This is also a good way to have singleton with minimum effort.

```

1  public enum EnumSingleton {
2      INSTANCE;
3
4      public void someMethod(String param) {
5          // some class member
6      }
7  }

```

Adding readResolve()

So, till now you must have taken your decision that how you would like to implement your singleton. Now lets see other problems that may arise even in interviews also.

Lets say your application is distributed and it frequently serialize the objects in file system, only to read them later when required. Please note that, de-serialization always creates a new instance. Lets understand using an example:

Our singleton class is:

```

1  public class DemoSingleton implements Serializable {
2      private volatile static DemoSingleton instance = null;
3
4      public static DemoSingleton getInstance() {
5          if (instance == null) {
6              instance = new DemoSingleton();
7          }
8          return instance;
9      }
10
11      private int i = 10;
12
13      public int getI() {
14          return i;
15      }
16
17      public void setI(int i) {
18          this.i = i;
19      }
20
21 }

```

Lets serialize this class and de-serialize it after making some changes:

```

1  public class SerializationTest {
2      static DemoSingleton instanceOne = DemoSingleton.getInstance();
3
4      public static void main(String[] args) {
5          try {
6              // Serialize to a file
7              ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(
8                  "filename.ser"));
9              out.writeObject(instanceOne);
10             out.close();
11
12             instanceOne.setI(20);
13
14             // Serialize to a file
15             ObjectInput in = new ObjectInputStream(new FileInputStream(
16                 "filename.ser"));
17             DemoSingleton instanceTwo = (DemoSingleton) in.readObject();
18             in.close();
19
20             System.out.println(instanceOne.getI());
21             System.out.println(instanceTwo.getI());
22
23             } catch (IOException e) {
24                 e.printStackTrace();
25             } catch (ClassNotFoundException e) {
26                 e.printStackTrace();
27             }
28         }
29     }
30 }

```

Output:

```

20
10

```

Unfortunately, both variables have different value of variable "i". Clearly, there are two instances of our class. So, again we are in same problem of multiple instances in application.

To solve this issue, we need to include readResolve() method in our DemoSingleton class. This method will be invoked when you will de-serialize the object. Inside this method, you must return the existing instance to ensure single instance application wide.

```

1  public class DemoSingleton implements Serializable {
2      private volatile static DemoSingleton instance = null;
3
4      public static DemoSingleton getInstance() {
5          if (instance == null) {
6              instance = new DemoSingleton();
7          }
8          return instance;
9      }
10
11      protected Object readResolve() {
12          return instance;
13      }
14
15      private int i = 10;
16
17      public int getI() {
18          return i;
19      }
20
21      public void setI(int i) {
22          this.i = i;
23      }
24 }

```

Now when you execute the class SerializationTest, it will give you correct output.

```

1  20
2  20

```

Adding serial version id

So far so good. Till now, we have solved the problem of synchronization and serialization both. Now, we are just one step behind our correct and complete implementation. And missing part is serial version id.

This is required in condition when you class structure can change in between you serialize the instance and go again to de-serialize it. Changed structure of class will cause JVM to give exception while de-serializing process.


```

1 java.io.InvalidClassException: singleton.DemoSingleton; local class incompatible
2 at java.io.ObjectStreamClass.initNonProxy(Unknown Source)
3 at java.io.ObjectInputStream.readNonProxyDesc(Unknown Source)
4 at java.io.ObjectInputStream.readClassDesc(Unknown Source)
5 at java.io.ObjectInputStream.readOrdinaryObject(Unknown Source)
6 at java.io.ObjectInputStream.readObject0(Unknown Source)
7 at java.io.ObjectInputStream.readObject(Unknown Source)
8 at singleton.SerializationTest.main(SerializationTest.java:24)

```

This problem can be solved only by adding a unique serial version id to class. It will prevent the compiler to throw the exception by telling that both classes are same, and will load the available instance variables only.

Conclusion

After having discussed so many possible approaches and other possible error cases, i will recommend you below code template to design your singleton class which shall ensure only one instance of class in whole application in all above discussed scenarios.

```

1 public class DemoSingleton implements Serializable {
2     private static final long serialVersionUID = 1L;
3
4     private DemoSingleton() {
5         // private constructor
6     }
7
8
9     private static class DemoSingletonHolder {
10         public static final DemoSingleton INSTANCE = new DemoSingleton();
11     }
12
13
14     public static DemoSingleton getInstance() {
15         return DemoSingletonHolder.INSTANCE;
16     }
17
18
19     protected Object readResolve() {
20         return getInstance();
21     }
22 }

```

I hope, this post has enough information to make you understand the most common approaches for singleton pattern. Let me know of you thoughts please.

Happy Learning !!

Update: I just thought to add some examples which can be referred for further study and mention in

interviews:

- java.awt.Desktop#getDesktop()
- java.lang.Runtime#getRuntime()